

CVA6 Documented medeleg[9] Missing Trap-Surface Bypass Vulnerability Report

Affected version

Issue package: ../../issues/CV-3

This package was validated on a local CVA6 checkout at revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the standard non-H cv64a6_imafdc_sv39 configuration. No fix revision has been identified yet.

Background

The issue package states the root bug: current non-H CVA6 documents medeleg[31:0] as RW with a bit position allocated for every synchronous exception, but omits medeleg[9] from the implemented writable mask. The issue PoC writes only medeleg[9], reads it back as clear, then observes an S-mode ecall trap to M-mode with tohost = 91.

The security PoC uses a firmware setup that follows the documented CVA6 CSR contract and expects S-mode ECALLs to enter an S-mode syscall gate, while reserving the M-mode trap path for violations or fatal handling.

Vulnerability type and CWE

Vulnerability type. Documented trap-delegation mechanism missing in the implementation, causing a trap-surface bypass for software that relies on the documented CVA6 CSR contract.

CWE. Primary: CWE-693 Protection Mechanism Failure. Secondary: CWE-755 Improper Handling of Exceptional Conditions and CWE-284 Improper Access Control.

Core defect

The CVA6-specific problem is that the in-tree documentation describes the medeleg CSR as implementing a writable bit for every synchronous exception, including bit 9:

- medeleg is listed as an RW CSR at 0x302;
- the medeleg field [31:0] is listed as RW;
- the documentation says there is a bit position for every synchronous exception, with the bit index equal to the mcause value.

The RTL has the corresponding trap-routing machinery, but the writable mask for CSR_MEDELEG omits ENV_CALL_SMODE:

- S-mode ecall decode assigns cause ENV_CALL_SMODE;
- synchronous trap routing checks medeleg_q[cause];
- the CSR_MEDELEG writable mask includes ENV_CALL_UMODE and, when RVH is enabled, ENV_CALL_VSMODE, but it does not include ENV_CALL_SMODE.

The documentation text says:

```
"``medeleg``", "0x302", "32", "RW", "0x00000000",  
"Machine Exception Delegation"
```

```
"[31:0]", "``Synchronous_Exceptions``", "Synchronous Exceptions",  
"RW", "0b0"
```

There is a bit position allocated for every synchronous exception, with the index of the bit position equal to the value returned in the mcause register.

The root cause is visible in the writable mask for CSR_MEDELEG:

```
riscv::CSR_MEDELEG: begin  
  if (CVA6Cfg.RVS) begin  
    mask = (1 << riscv::INSTR_ADDR_MISALIGNED) |  
            (1 << riscv::INSTR_ACCESS_FAULT) |  
            (1 << riscv::ILLEGAL_INSTR) |  
            (1 << riscv::BREAKPOINT) |  
            (1 << riscv::LD_ADDR_MISALIGNED) |  
            (1 << riscv::LD_ACCESS_FAULT) |  
            (1 << riscv::ST_ADDR_MISALIGNED) |  
            (1 << riscv::ST_ACCESS_FAULT) |  
            (1 << riscv::ENV_CALL_UMODE) |  
            ((CVA6Cfg.RVH ? 1 : 0) << riscv::ENV_CALL_VSMODE) |  
            (1 << riscv::INSTR_PAGE_FAULT) |  
            (1 << riscv::LOAD_PAGE_FAULT) |  
            (1 << riscv::STORE_PAGE_FAULT) |  
            ((CVA6Cfg.RVH ? 1 : 0) << riscv::INSTR_GUEST_PAGE_FAULT) |  
            ((CVA6Cfg.RVH ? 1 : 0) << riscv::LOAD_GUEST_PAGE_FAULT) |  
            ((CVA6Cfg.RVH ? 1 : 0) << riscv::VIRTUAL_INSTRUCTION) |  
            ((CVA6Cfg.RVH ? 1 : 0) << riscv::STORE_GUEST_PAGE_FAULT);  
    medeleg_d = (medeleg_q & ~mask) | (csr_wdata & mask);  
  end  
end
```

ENV_CALL_SMODE is produced by the S-mode ecall decode path:

```
if (priv_lvl_i == riscv::PRIV_LVL_S && CVA6Cfg.RVS) begin  
  instruction_o.ex.cause =  
    (CVA6Cfg.RVH && v_i) ? riscv::ENV_CALL_VSMODE  
      : riscv::ENV_CALL_SMODE;  
end
```

Trap routing is already generic over medeleg_q[cause]:

```
if ((ex_i.cause[CVA6Cfg.XLEN-1] && medeleg_q[ex_i.cause[$clog2(  
  CVA6Cfg.XLEN  
)-1:0])) || (~ex_i.cause[CVA6Cfg.XLEN-1] && medeleg_q[ex_i.cause[$clog2(  
  CVA6Cfg.XLEN  
)-1:0])) begin  
  trap_to_priv_lvl =  
    (priv_lvl_o == riscv::PRIV_LVL_M) ? riscv::PRIV_LVL_M  
      : riscv::PRIV_LVL_S;  
end
```

Security relevance

The security setting is an M-mode compartment monitor that trusts the CVA6 manual and configures medeleg[9] so S-mode ECALLs are handled by an S-mode syscall gate. The monitor's policy is:

1. expected tenant service requests enter s_syscall_gate;
2. unexpected M-mode traps enter m_security_violation;
3. S-mode tenants must not be able to trigger the M-mode violation path merely by issuing an ordinary S-mode ecall.

With the documented CSR behavior, the tenant's ecall is handled horizontally in S-mode. On affected CVA6, medeleg[9] is masked out, so the same tenant-controlled ecall enters the M-mode trap path.

This creates an unexpected M-mode trap surface. In a real monitor this can map to denial of service through a fatal monitor panic, forced compartment reset, unexpected audit/lockout behavior, or integrity risk if the M-mode fallback path contains management operations that were not intended to be tenant-reachable.

Security PoC

Assumptions

Trusted M-mode code configures the documented `medeleg[9]` path to route S-mode ECALLs to a local S-mode gate.

PoC setup

The PoC installs two trap targets:

- `stvec` = `s_syscall_gate`, the expected S-mode handler;
- `mtvec` = `m_security_violation`, the M-mode violation path.

M-mode then writes:

```
li    t0, MEDELEG_ECALL_S
csrw medeleg, t0
```

The security PoC uses the documented CVA6 CSR behavior directly. After setting `mstatus.MPP = S`, the monitor enters S-mode at `s_tenant_entry` and the tenant executes:

```
s_ecall_site:
    ecall
```

Security result

- The monitor configures the documented `medeleg[9]` path.
- The S-mode tenant issues one ordinary `ecall`.
- The expected documented path is `s_syscall_gate`, which records `S_GATE_MARKER` and exits with code `0`.
- CVA6 instead enters `m_security_violation`, which records `M_PANIC_MARKER` and exits with code `31`.

CVA6 evidence

The evidence log is `poc/poc.cva6.log`:

```
poc.elf *** FAILED *** (tohost = 31) after 480 cycles
*** [rvfi_tracer] INFO: Simulation terminated after      468 cycles!
```

`tohost = 31` is the PoC's `m_security_violation` path. That path is reached only if the S-mode `ecall` traps to M-mode instead of the configured S-mode syscall gate.

Expected result

The S-mode `ecall` traps horizontally to `s_syscall_gate` with `scause = 9` and `sepc = s_ecall_site`. The S gate records `S_GATE_MARKER` and exits with `tohost = 0`.

Observed result on CVA6

CVA6 exits with `tohost = 31`, meaning the tenant-controlled S-mode `ecall` entered the M-mode security-violation handler.

Impact

- Primary: Availability. A tenant-controlled S-mode `ecall` can drive the monitor into an unexpected M-mode violation or panic path when software relied on the documented delegation feature.
- Secondary: Integrity. If the M-mode fallback path contains management, policy, audit, or recovery behavior that is not designed to be tenant-reachable, the missing delegation bit can expose that path to S-mode input.

Affected version status

Confirmed affected in a local run of CVA6 revision `v5.3.0-168-gcd4c8b17`, confirmed at commit `cd4c8b17c102`, using the `cv64a6_imafdc_sv39` configuration.

Fixed: none identified yet.

Fix direction

Include `ENV_CALL_SMODE` in the writable `CSR_MEDELEG` mask for RVS configurations and keep the issue PoC plus the security PoC as regression tests.